

UNIVERSIDAD NACIONAL JORGE BASADRE GROHMANN

FACULTAD DE INGENIERÍA



Escuela Profesional de Ingenieria en Informatica y Sistemas

Arquitectura de Computadoras

Docente: Mg. Genyfer Aldana S.

Presentado por:

Angel Aníbal Mamani Castilla

Diego Jeancarlos Ramos Marca

José Antonio Vilcanqui Chambi

Brayhan Deyvid Quispe Cama

TACNA-2026

1. Introducción

Resumen del Proyecto

En este proyecto implementamos, en Logisim Evolution, un simulador digital del ciclo de instrucción de un procesador simplificado. El circuito reproduce las tres etapas clásicas que estudiamos en el curso: la búsqueda de la instrucción en memoria (fetch), su interpretación mediante un decodificador (decode) y la ejecución de la operación indicada (execute). Para lograrlo diseñamos, con registros, multiplexores, sumadores y compuertas lógicas, un datapath capaz de ejecutar cuatro instrucciones básicas: ADD, LOAD, STORE y JUMP.

El control del ciclo lo manejamos de forma manual con tres interruptores (A, B y C), que en un procesador real corresponden a señales generadas automáticamente por la unidad de control. Usarlos manualmente nos permitió, además, entender con mayor claridad qué señal hace qué y en qué momento exacto del ciclo actúa.

Justificación

Entender el ciclo de instrucción es la base para comprender cómo funciona cualquier procesador, desde uno académico de 8 bits hasta una arquitectura moderna x86 o ARM. Toda CPU, sin importar su complejidad, repite constantemente el patrón fetch-decode-execute; lo único que cambia entre arquitecturas es qué tan optimizada, segmentada (pipeline) o paralelizada está cada etapa. Al construir manualmente cada registro, cada multiplexor y cada señal de control, dejamos de ver la arquitectura del computador como una caja negra y entendemos por qué

existen conceptos como el registro de instrucción, el direccionamiento a memoria o la decodificación de opcodes.

1. Objetivos

Objetivo General

Demostrar el funcionamiento del ciclo de instrucción (fetch-decode-execute) mediante un simulador digital construido en Logisim Evolution.

Objetivos Específicos

- Diseñar un datapath con registros (PC, MAR, MDR, IR, AC) y una memoria RAM capaz de almacenar y ejecutar un pequeño programa de prueba.
- Implementar la lógica de control necesaria (multiplexores e interruptores) para alternar entre la etapa de búsqueda de instrucción y la etapa de ejecución del operando.
- Evaluar el rendimiento del circuito en términos de ciclos de reloj necesarios por instrucción y verificar su correcto funcionamiento mediante pruebas con distintas instrucciones.

2. Marco Teórico

El ciclo de instrucción es la secuencia de pasos que ejecuta un procesador para procesar cada instrucción de un programa. Básicamente se divide en fetch (búsqueda), decode (decodificación) y execute (ejecución).

En la etapa de fetch, el Program Counter (PC) contiene la dirección de la siguiente instrucción a ejecutar. Esa dirección se copia al Memory Address Register (MAR), la memoria entrega el contenido de esa dirección, y ese contenido se guarda en el Memory Data Register (MDR) para luego pasar al Instruction Register (IR). Al finalizar esta etapa, el PC se incrementa para apuntar a la siguiente instrucción.

En la etapa de decode, el opcode contenido en el IR se interpreta mediante un decodificador binario, que activa una única línea de salida correspondiente a la operación a realizar (en nuestro caso ADD, LOAD, STORE o JUMP).

En la etapa de execute, el operando (la otra mitad de la instrucción) se usa como dirección de memoria o como dato, según la operación decodificada, y el resultado se guarda en el registro correspondiente (el acumulador AC, la memoria RAM o el propio PC).

Un concepto clave aplicado en este proyecto es el de modos de direccionamiento: nuestras instrucciones usan direccionamiento directo, ya que el nibble bajo de la instrucción es directamente la dirección de memoria del operando, sin niveles adicionales de indirección.

3. Desarrollo del Proyecto (Parte Práctica)

El proyecto lo desarrollamos íntegramente en Logisim Evolution, siguiendo el método de simulación digital sugerido para proyectos de tipo CPU (registro 06 del cuadro de aplicación técnica del curso).

4.1 Arquitectura general del circuito

El datapath que construimos está compuesto por los siguientes bloques principales:

- PC (Program Counter): registro que guarda la dirección de la próxima instrucción. Tiene un sumador asociado (+1) para autoincrementarse.
- MAR (Memory Address Register): registro que guarda la dirección que se va a consultar en la RAM, ya sea para leer una instrucción o un operando.

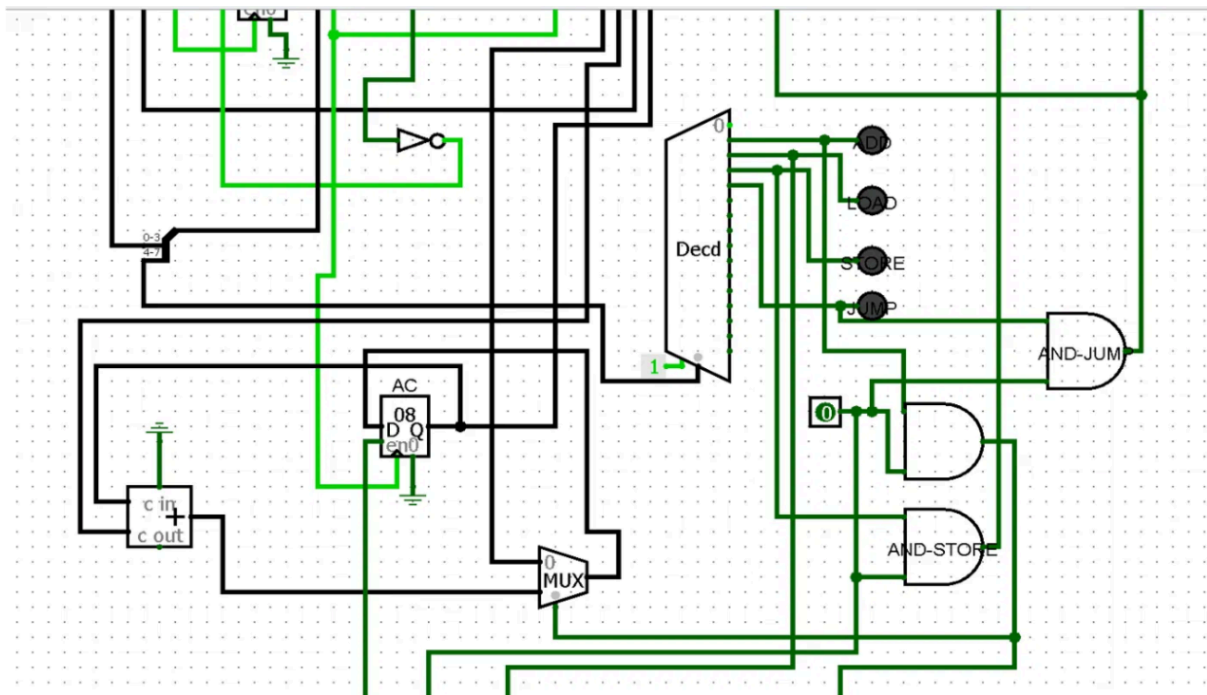


Figura 2. Decodificador de opcode y bloque de ejecución (AC, compuertas AND/OR).

4.2 Etapa de Fetch

Durante el fetch, el interruptor A permanece en 0, de modo que el MUX del MAR selecciona la dirección proveniente del PC. Esa dirección entra a la RAM, cuya salida pasa al MDR y luego al IR. En el primer pulso de reloj del fetch activamos también el interruptor C, para que el PC avance ($PC+1$) y quede listo apuntando a la siguiente instrucción del programa, sin afectar el resto del ciclo.

Por ejemplo, en la dirección 0x0 de nuestra memoria de prueba se encuentra el valor 2A: el IR toma este byte y el splitter lo separa en el nibble alto (2, el opcode) y el nibble bajo (A, el operando/dirección).

4.3 Etapa de Decode

Una vez que el IR contiene la instrucción, el nibble alto (el opcode) se conecta a la entrada del decodificador. El decodificador activa una única salida entre las 16 posibles, de las cuales solo usamos cuatro: ADD, LOAD, STORE y JUMP. Esa señal activa es la que habilita, más adelante, la compuerta AND correspondiente a la operación.

Para STORE y JUMP usamos compuertas AND dedicadas (AND-STORE y AND-JUMP) que combinan la señal del decodificador con el interruptor B. Para ADD y LOAD combinamos sus respectivas señales de decodificación mediante una compuerta OR, ya que ambas instrucciones terminan habilitando la escritura en el mismo registro (el acumulador AC).

4.4 Etapa de Execute

Al iniciar la ejecución activamos el interruptor A ($A = 1$), con lo cual el MUX del MAR deja de tomar la dirección del PC y empieza a tomar el nibble bajo del IR (el operando). De esta forma el MAR apunta directamente a la dirección de memoria que la instrucción indica como operando.

- ADD: la RAM entrega el dato de esa dirección, pasa por el MDR, se suma al valor actual del AC mediante el sumador, y el resultado se guarda en el AC cuando activamos el interruptor B.
- LOAD: el dato leído de la RAM se copia directamente al AC (a través del MUX del AC) al activar el interruptor B.
- STORE: el valor actual del AC se escribe en la RAM, en la dirección indicada por el operando, activando la línea str de la memoria mediante la compuerta AND-STORE y el interruptor B.

- JUMP: el operando del IR se carga directamente al PC (en vez de PC+1), activando la compuerta AND-JUMP junto con el interruptor B, lo que provoca un salto en el flujo del programa.

Terminada la ejecución, regresamos el interruptor A a 0 para volver al modo fetch y buscar la siguiente instrucción.

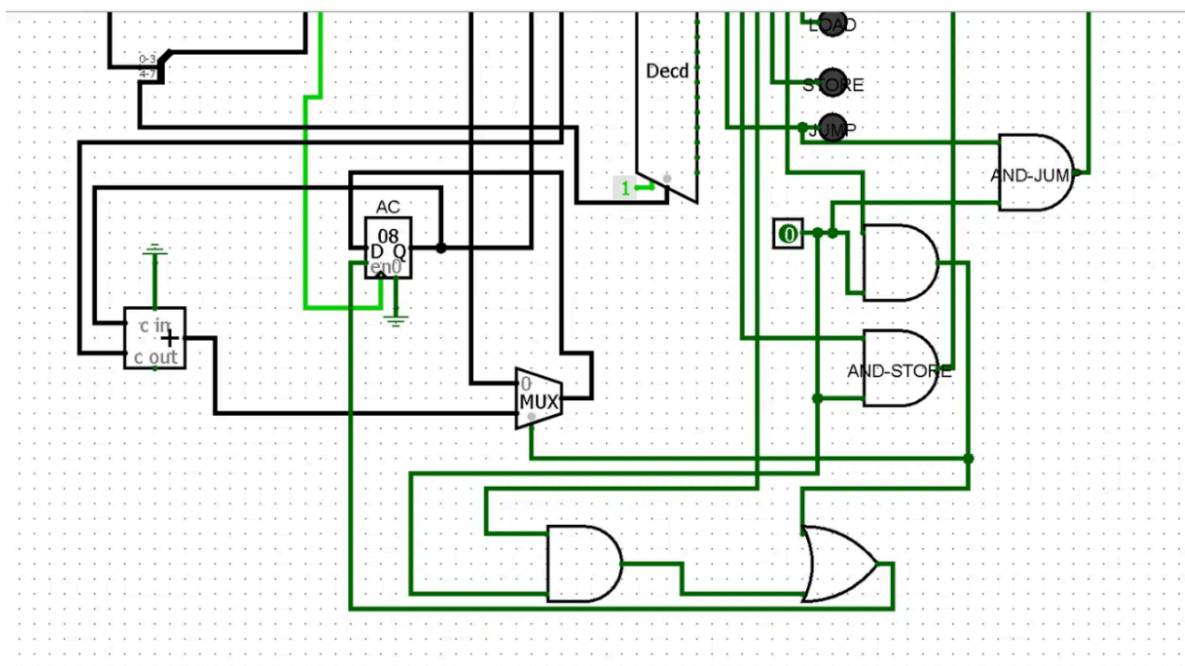


Figura 3. Detalle de las compuertas AND-STORE, AND-JUMP, la compuerta OR (ADD/LOAD) y el MUX del acumulador.

4.5 Los tres interruptores de control

Como no implementamos una unidad de control automática, el ciclo completo lo manejamos con tres interruptores, cada uno con una función y un momento de uso muy específico:

Interruptor A — Selector Fetch/Execute

- Controla: la entrada sel del MUX del MAR.

- $A = 0$: el MAR recibe la dirección desde el PC (modo fetch, se busca la siguiente instrucción).
- $A = 1$: el MAR recibe la dirección desde el splitter del IR (modo execute, se busca el operando de la instrucción actual).
- Se activa justo al iniciar el execute de cualquier instrucción, y se regresa a 0 al terminar, para volver al modo fetch.

Interruptor B — Habilitar grabado/escritura

- Controla, a través de las compuertas AND/OR: el en del AC (ADD y LOAD), el str de la RAM (STORE) y el Id del PC (JUMP).
- Activa el instante exacto en que el resultado final de la instrucción se guarda, ya sea en el AC, en la memoria o en el PC.
- Se usa solo en el último pulso del execute, después de que el MAR (y el MDR, si aplica) ya tienen el valor correcto. Funciona como el "confirmar" de la operación.

Interruptor C — Habilitar avance del PC

- Controla: el ct (count enable) del PC.
- $C = 1$: el PC avanza ($PC+1$) en el próximo pulso. $C = 0$: el PC se congela.
- Se usa solo en el primer pulso del fetch, cuando el MAR toma el valor actual del PC, para que el PC quede apuntando a la siguiente dirección sin interferir con el resto del ciclo.

4.6 Uso de los multiplexores (MUX)

En el circuito usamos tres multiplexores, cada uno resolviendo una decisión distinta del datapath:

- MUX del PC: decide si el PC mantiene su valor actual o recibe el resultado del sumador ($PC+1$) o el operando del IR en un JUMP.
- MUX del MAR: es el que controla el interruptor A; decide si la dirección que se consulta en la RAM proviene del PC (fetch) o del IR (execute).
- MUX del AC: decide si el acumulador recibe el resultado de la suma (ADD) o el dato leído directamente de memoria (LOAD).

En conjunto, estos multiplexores son los que permiten que un mismo datapath físico sirva tanto para la etapa de fetch como para las cuatro operaciones distintas de la etapa de execute, sin necesidad de duplicar registros ni buses.

5. Pruebas y Resultados

Tabla de Pruebas

Dirección	Instrucción (hex)	Opcode	Operando	Resultado esperado
0x0	2A	2	A (0xA)	Se decodifica y ejecuta según el opcode 2; el operando A indica la dirección del dato en RAM.
0x1	1B	1	B (0xB)	Se decodifica y ejecuta según el opcode 1; el operando B indica la dirección del dato en RAM.
0x2	3D	3	D (0xD)	Se decodifica y ejecuta según el opcode 3; el operando D indica la dirección del dato en RAM.
0x3	4F	4	F (0xF)	Se decodifica y ejecuta según el opcode 4; el operando F indica la dirección destino (JUMP) o del dato en RAM.

Análisis de Rendimiento

Cada instrucción nos tomó 5 pulsos de reloj: 3 para el fetch (que es siempre igual, no importa la instrucción) y 2 para el execute. La única que se sale de ese patrón es

JUMP, que resuelve todo en un solo pulso de execute porque no necesita leer nada de la RAM — el operando ya está en el IR y pasa directo al PC. Con esto confirmamos algo bastante intuitivo: acceder a memoria cuesta ciclos, y una instrucción que se salta ese paso es más rápida.

Análisis de Errores

El bug que más nos costó encontrar fue el del IR: al principio dejamos su entrada *en* fija en 1, así que en cada pulso del execute el registro se sobrescribía con lo que fuera que hubiera en el MDR, perdiendo el opcode a medio camino. Lo arreglamos condicionando $en_IR = NOT(Interruptor\ A)$.

El error más grande fue con JUMP: el PC estaba armado como Counter, y ese tipo de componente no deja cargarle un valor cualquiera por D salvo que haya overflow — por más bien diseñada que estuviera la lógica del salto, el PC simplemente seguía contando. Tuvimos que cambiarlo por un Register de 4 bits, igual que los demás registros. Y ya con eso resuelto, nos topamos con un efecto secundario: el PC se incrementaba en los tres pulsos del fetch en vez de solo uno, porque su *en* dependía nada más de $NOT(Interruptor\ A)$. La solución fue meter un control manual aparte para ese avance, combinado con la señal de JUMP mediante una OR.

6. Conclusiones y Recomendaciones

Conclusiones

- Cumplimos el objetivo general: el simulador reproduce correctamente las tres etapas del ciclo de instrucción y ejecuta las cuatro operaciones propuestas (ADD, LOAD, STORE, JUMP).
- Como grupo, comprendimos que la "unidad de control" de un procesador real no es más que la generación automática, en el momento correcto, de señales equivalentes a nuestros interruptores A, B y C; nosotros simplemente las reemplazamos por control manual.
- También reforzamos el concepto de que un mismo datapath puede reutilizarse para múltiples instrucciones gracias a los multiplexores, en vez de necesitar un circuito distinto por cada operación.

Recomendaciones

- Para una siguiente versión, se podría reemplazar los interruptores manuales por una máquina de estados finitos (FSM) que genere automáticamente las señales A, B y C según el ciclo de reloj.
- Ampliar el decodificador para aprovechar las 16 salidas disponibles y soportar más instrucciones (por ejemplo, SUB o instrucciones condicionales).

7. Bibliografía

Stallings, W. (2019). Organización y arquitectura de computadores (10.^a ed.). Pearson Educación.

Patterson, D. A., & Hennessy, J. L. (2020). Computer Organization and Design: The Hardware/Software Interface (5.^a ed.). Morgan Kaufmann.

9. Anexos

Archivo logisim:

 CicloFetchDecoderExecute

Enlace al video de demostración:

<https://youtu.be/tOwHRvW73wo>